WEEK 3

Web Application Testing Lab

Name: Vijayashree B
Domain: Cybersecurity (VAPT)
Date of Submission: 30.03.26

Web Application Security Testing Report

1. Introduction

This lab focuses on testing a deliberately vulnerable web application (DVWA – Damn Vulnerable Web Application) to identify common security flaws listed in the OWASP Top 10. The objective is to perform both manual and automated testing using industry-standard tools and document the findings.

2. Objectives

- Identify common web vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS)
- Understand how attackers exploit insecure input handling
- Use tools like Burp Suite, sqlmap, and OWASP ZAP for testing
- Analyze and document vulnerabilities with severity levels
- Recommend mitigation techniques

3. Test Environment

- **Target Machine (DVWA):** 192.168.56.101
- **Attacker Machine (Kali Linux):** 192.168.56.105
- **Tools Used:**
  - Burp Suite (Proxy & Interception)
  - sqlmap (SQL Injection Automation)
  - OWASP ZAP (Automated Scanner)
- **Environment Type:** Isolated virtual lab network

4. Test Log

Test ID	Vulnerability	Severity	Target URL
001	SQL Injection	Critical	http://192.168.56.101/login
002	XSS Reflected	Medium	http://192.168.56.101/form

5. Methodology

5.1 Manual Testing (Burp Suite)

- Intercepted HTTP requests using proxy
- Modified parameters in requests
- Tested authentication bypass techniques
- Captured cookies and session tokens

5.2 Automated Testing (sqlmap)

- Used intercepted request for SQL injection testing
- Enumerated database structure
- Extracted database names

5.3 Automated Scanning (OWASP ZAP)

- Crawled target application
- Performed active scan
- Identified vulnerabilities and categorized them by severity

6. Detailed Findings

6.1 SQL Injection (Critical)

- **Location:** Login page
- **Payload Used:** `' OR '1'='1`
- **Impact:**
  - Authentication bypass achieved
  - Unauthorized access granted
  - sqlmap successfully extracted database names
- **Evidence:**
  - Login bypass using crafted input
  - sqlmap output showing available databases
- **Risk:** Full database compromise

6.2 Reflected Cross-Site Scripting (XSS) (Medium)

- **Location:** Input form
- **Payload Used:**
  `<script>alert('XSS')</script>`
- **Impact:**
  - Script executed in browser
  - Popup alert confirms vulnerability
  - Can lead to session hijacking or phishing
- **Evidence:**
  - Browser alert displaying injected script

6.3 OWASP ZAP Findings

- **High Risk:** SQL Injection
- **Medium Risk:** Cross-Site Scripting

- **Low Risk:** Missing security headers
- **Informational:** Cookie security issues

7. OWASP ZAP Analysis

7.1 Dashboard Overview

The dashboard provided a visual summary of vulnerabilities categorized by severity (High, Medium, Low, Informational). It highlighted multiple endpoints affected by insecure coding practices.

7.2 Alerts Summary

- SQL Injection vulnerability detected in login functionality
- XSS vulnerability identified in input fields
- Missing HTTP security headers observed
- Cookies lacking secure flags detected

8. Checklist

- Tested SQL Injection using sqlmap
- Performed manual XSS testing
- Intercepted requests with Burp Suite
- Verified authentication bypass
- Analyzed results using OWASP ZAP
- Reviewed session and cookie security

9. Recommendations

- Use prepared statements (parameterized queries) to prevent SQL injection
- Implement input validation and output encoding
- Apply Content Security Policy (CSP)
- Enable HttpOnly and Secure flags on cookies
- Use web application firewalls (WAF)
- Regularly perform security testing

10. Conclusion

The testing of the DVWA application revealed critical security vulnerabilities, including SQL Injection and Reflected XSS. These flaws demonstrate poor input validation and lack of secure coding practices. Both manual and automated tools confirmed the presence of exploitable weaknesses, emphasizing the need for proper security controls.

11. 50-Word Summary

The DVWA web application was tested for OWASP Top 10 vulnerabilities using Burp Suite, sqlmap, and OWASP ZAP. Critical SQL injection enabled authentication bypass and database

extraction. Reflected XSS allowed script execution in the browser. These issues highlight weak input validation and inadequate security mechanisms in the application.